Workflow Templates — Wyniki Testów v2 (double-check)

Dokument zawiera wyniki drugiej rundy testów przeprowadzonych z perspektywy eksperta SEM/analityki. Każdy test oparty na bezpośrednim odczycie pliku JSON szablonu.

Cel: porównanie z pierwszą rundą testów (`workflow-templates-test-results.md`) w celu weryfikacji poprawności wyników.

Legenda:

- 🤖 **Auto** — wartość generowana przez system
- 👤 **User** — wartość którą użytkownik musi uzupełnić
- 📋 **Template** — wartość zakodowana w szablonie
- ✅ węzeł działa poprawnie
- ⚠️ węzeł działa ale z zastrzeżeniem
- ❌ węzeł nie działa / błąd

Test #1 — `cpa_campaign_alert` *(Google Ads)*

Data testu: 2026-03-30 | **Plik:** `cpa_campaign_alert.json`

Węzeł 1 — Trigger

- ✅ Nowy format (`schedule`)
- 📋 `frequency: weekly`, `days_of_week: monday`, `time: 08:00`
- ✅ `timezone: Europe/Warsaw`

Ocena SEM: Tygodniowy monitoring CPA to właściwa częstotliwość — Google Ads aktualizuje konwersje z opóźnieniem 24–48h, więc sprawdzanie rzadziej niż daily nie traci wartości przy oknie 30 dni.

Węzeł 2 — Getter: `get_google_campaign`

- ✅ `campaign_statuses: ["ENABLED"]` — tylko aktywne kampanie
- 👤 `connection_id: null`, `project_id: null`

Węzeł 3 → 4 — Iterator + `get_google_ads_metrics`

- ✅ `resourceId: {3.campaignId}` — metryki per kampania
- ✅ Okno 30 dni — statystycznie istotne dla CPA
- ✅ `limit: 1`

Węzeł 5 — Filtr (3 warunki)

```
(1) cost > {globals.min_spend}                        [AND]
(2) conversions > 0                                   [AND]
(3) divide(cost,conversions) > {globals.cpa_target}   [OR]  ← ostatni warunek
```

⚠️ **Błąd logiczny:** ostatni warunek ma `logical_operator: "OR"` zamiast `"AND"`. W praktyce bez efektu (brak kolejnego warunku), ale jako wzorzec kodu jest mylący i może powodować błędy po przyszłych modyfikacjach.

✅ **Warunek** `conversions > 0` — chroni przed dzieleniem przez zero. Dobrze zaprojektowane.

Dane testowe (zmyślone, format CampaignDTO + MetricsDTO):

Kampania	cost	conversions	CPA	cost>100?	conv>0?	CPA>50?	Przechodzi?
Brand Exact	1240	48	25.83	✅	✅	❌	❌
Non-Brand	3800	29	131.03	✅	✅	✅	✅
Shopping	5200	118	44.07	✅	✅	❌	❌
Remarketing	340	0	∞	✅	❌	—	❌
Display	89	1	89.00	❌	✅	✅	❌

Wynik: 1 kampania przechodzi (Non-Brand).

Węzeł 6 — Iterator (sourceId: 4)

⚠️ **Redundantny węzeł.** Node 4 zwraca `limit: 1` — zawsze 1 element. Iterator po 1-elementowej tablicy jest zbędny i dodaje niepotrzebną warstwę złożoności.

Węzeł 7 — `create_task`

Task:

> ***Tytuł:*** *`🚨 CPA Alert: Non-Brand – CPA: 131.03448... (cel: 50)`*

⚠️ **Brak zaokrąglenia CPA.** `{divide({4.cost},{4.conversions})}` zwróci pełną precyzję zmiennoprzecinkową (np. `131.03448275862069`). W tytule taska wygląda nieprofesjonalnie.

⚠️ **CTR w opisie:** `{4.ctr}%` — jeśli getter zwraca CTR jako dziesiętne (0.032), użytkownik zobaczy `0.032%` zamiast `3.2%`. Wymaga weryfikacji formatu DTO.

⚠️ **`{4.conversions_value}`** — pole w opisie taska. Należy zweryfikować czy `get_google_ads_metrics` faktycznie zwraca to pole.

✅ **Bogaty opis taska** — koszt, konwersje, CPA, odchylenie od celu, sugerowane działania.

Ocena eksperta SEM

Luka analityczna: Szablon nie sprawdza `biddingStrategyType`. Kampanie z `MAXIMIZE_CLICKS` lub `TARGET_IMPRESSION_SHARE` nie optymalizują konwersji — ich CPA może być naturalnie wysoki lub niski bez związku z jakością kampanii. Alert może generować fałszywe alarmy dla tych typów.

Brakująca metryka: Brak ROAS w opisie taska. Dla e-commerce ROAS często ważniejszy niż CPA — kampania z CPA 131 PLN ale ROAS 4.5 może być rentowna.

Podsumowanie #1

Status	✅ działa end-to-end
Taski utworzone	1 (Non-Brand przekracza CPA)
Znane problemy	1. Redundantny węzeł 6 (iterator po 1 elemencie). 2. CPA bez zaokrąglenia w tytule. 3. `logical_operator: "OR"` na ostatnim warunku — mylący. 4. CTR format niezweryfikowany (dziesiętne vs %). 5. Brak filtra na `biddingStrategyType`.
Ocena ogólna	Solidny szablon alertowy — nowy format, konfigurowalny, zabezpieczony przed /0. Drobne problemy UX (formatowanie) i analityczne (brak filtra na typ strategii).
Rekomendacja	Usunąć węzeł 6 (redundantny iterator). Dodać `round()` do CPA w tytule. Zmienić `logical_operator` na "AND". Rozważyć dodanie ROAS do opisu.

Test #2 — `roas_drop_alert` *(Google Ads)*

Data testu: 2026-03-30 | **Plik:** `roas_drop_alert.json`

Węzeł 1 — Trigger

- ✅ Nowy format (`schedule`), weekly poniedziałek 08:00
- ✅ `timezone: Europe/Warsaw`

Węzły 2–3 — Getter + Iterator

- ✅ `campaign_statuses: ["ENABLED"]`
- 👤 `connection_id: null`, `project_id: null`

Węzły 4–5 — Dual Getter (dwa okresy 7-dniowe)

- Node 4: ostatnie 7 dni (`subtractNumberOfDays(7)` → dziś)
- Node 5: poprzednie 7 dni (`subtractNumberOfDays(14)` → `subtractNumberOfDays(7)`)
- ✅ Porównanie dwóch tygodni — wykrywa trend, nie tylko absolutny poziom

⚠️ **ROAS obliczany jako `divide(conversions_value, cost)`** — wymaga żeby `conversions_value` było dostępne w MetricsDTO. Jeśli kampania nie ma skonfigurowanego śledzenia wartości konwersji, ROAS = 0 i alert zawsze się odpali.

Węzeł 6 — Filtr

```
(1) cost(node4) > {globals.min_spend}            [AND]
(2) divide(conversions_value,cost) < roas_target  [OR]  ← OR na ostatnim warunku
```

⚠️ **Brak warunku `conversions_value > 0`** — kampanie bez wartości konwersji mają ROAS=0, zawsze < 3.0 → fałszywy alarm.

Dane testowe:

Kampania	cost (7d)	conv_val (7d)	ROAS	cost>100?	ROAS<3?	Przechodzi?
Brand Exact	1180	5200	4.41	✅	❌	❌
Non-Brand	3600	6120	1.70	✅	✅	✅
Shopping	4900	22050	4.50	✅	❌	❌
Remarketing	290	0	0	✅	✅	✅ ← fałszywy alarm

Wynik: 2 kampanie przechodzą — 1 słuszna, 1 fałszywy alarm (brak conv_value).

Węzeł 7 — Iterator (sourceId: 4)

⚠️ **Redundantny** — `limit: 1` na getterze, iterator po 1 elemencie zbędny.

Węzeł 8 — `create_task`

✅ Bardzo bogaty opis — oba okresy, percentageChange, 5 sugerowanych działań. ⚠️ **Nazwa "ROAS Drop Alert" myląca** — wykrywa ROAS poniżej progu, nie spadek. Kampania z permanentnie niskim ROAS alarmuje co tydzień. ⚠️ **Brak zaokrąglenia ROAS** w tytule.

Ocena eksperta SEM

Okno 7+7 dni podatne na sezonowość tygodniową — dla e-commerce lepsze 14+14 lub 30+30 dni. Brak kontekstu strategii stawek w opisie taska (TARGET_ROAS vs MAXIMIZE_CONVERSIONS mają różną interpretację).

Podsumowanie #2

Status	✅ działa end-to-end
Taski utworzone	2 (1 słuszny + 1 potencjalnie fałszywy)
Znane problemy	1. Brak `conversions_value > 0` — fałszywe alarmy. 2. Redundantny węzeł 7. 3. Nazwa "Drop Alert" myląca. 4. `logical_operator: "OR"` na ostatnim warunku. 5. ROAS bez zaokrąglenia.
Ocena ogólna	Dobry szablon z wartościowym dual-getter porównaniem. Główna luka: fałszywe alarmy dla kampanii bez `conversions_value`.
Rekomendacja	Dodać `conversions_value > 0`. Zmienić nazwę na "ROAS Below Target Alert". Usunąć redundantny iterator. Rozważyć 14+14 dni.

Test #3 — `zero_impression_campaigns` *(Google Ads)*

Data testu: 2026-03-30 | **Plik:** `zero_impression_campaigns.json`

Węzeł 1 — Trigger

- ✅ Nowy format, weekly poniedziałek 08:00, `timezone: Europe/Warsaw`

Węzły 2–3 — Getter + Iterator

- ✅ `campaign_statuses: ["ENABLED"]`
- 👤 `connection_id: null`, `project_id: null`
- `globals: {}` — brak parametrów konfigurowalnych (właściwe dla tej logiki)

Węzeł 4 — `get_google_ads_metrics`

- ✅ Okno 7 dni, `resourceId: {3.campaignId}`, `limit: 1`

Węzeł 5 — Filtr

```
impressions(node4) < 1   [OR]
```

✅ Prosta, jednoznaczna logika. `< 1` łapie dokładnie 0 wyświetleń.

Węzeł 6 — Iterator (sourceId: 4)

⚠️ **Redundantny** — ten sam problem co #1 i #2. `limit: 1`, iterator zbędny.

Węzeł 7 — `create_task`

✅ Bogaty opis z 6-punktową listą przyczyn + link do diagnostyki Google Ads.

Dane testowe:

Kampania	impressions (7d)	impressions < 1?	Przechodzi?
Brand Exact	9400	❌	❌
Non-Brand	28500	❌	❌
Shopping	0	✅	✅
New Summer 2026	0	✅	✅

Wynik: 2 kampanie dostają task.

Ocena eksperta SEM

⚠️ **Fałszywe alarmy — nowe kampanie:** kampania uruchomiona w trakcie tygodnia może mieć 0 wyświetleń podczas weryfikacji reklam (24–72h). Brak filtra na `campaign_start_date`. ⚠️ **Fałszywe alarmy — harmonogram reklam:** kampania "tylko weekendy" sprawdzana w tygodniu bez weekendu (święta) może dać 0.

Podsumowanie #3

Status	✅ działa end-to-end
Taski utworzone	2 (Shopping + nowa kampania)
Znane problemy	1. Redundantny węzeł 6. 2. Fałszywe alarmy dla nowych kampanii. 3. Fałszywe alarmy dla kampanii z harmonogramem reklam.
Ocena ogólna	Prosty i użyteczny szablon. Bogaty opis diagnostyczny. Fałszywe alarmy możliwe ale rzadkie w typowym koncie.
Rekomendacja	Usunąć redundantny iterator. Dodać wzmiankę o nowych kampaniach i harmonogramie w opisie szablonu.

Test #4 — `budget_protector` *(Google Ads)*

Data testu: 2026-03-30 | **Plik:** `budget_protector.json`

Węzeł 1 — Trigger

- ❌ Stary format (`trigger_type: cron`, `cron: "0 "`) — co godzinę, 24×/dobę
- ❌ Brak timezone
- ❌ `status: "active"` — szablon domyślnie **włączony** — natychmiastowy task flood po imporcie

Węzeł 2 — Getter

- ❌ `connection_id: ""`, `project_id: ""` — puste stringi
- ❌ Brak `campaign_statuses` — pobiera ENABLED + PAUSED + REMOVED
- ❌ `parent_id: null` — węzeł odłączony od triggera

Węzeł 3 — Iterator

- ❌ Stary format (`flow_type: iterator`)

Węzeł 4 — `get_google_ads_metrics`

- ❌ `startDate` / `endDate` camelCase zamiast `start_date` / `end_date`
- ⚠️ Zakres: wczoraj+dziś (ostatnie 24h), nie "tylko dziś" → koszt zawyżony

Węzeł 5 — Filtr

- ❌ Stary format filtra
- ❌ `globals: []` — tablica zamiast obiektu, próg `cost > 100` hardcoded

Węzeł 6 — `create_task`

- ❌ Stary format, brak `project_id`, minimalny opis taska

Symulacja — 3 kampanie > 100 PLN przez dobę = 45 tasków/dobę (150 przy 10 kampaniach)

Ocena eksperta SEM

Najbardziej szkodliwy szablon w zestawie — domyślnie aktywny, generuje task flood od pierwszej minuty. Koncepcja intraday monitoringu jest wartościowa, ale wymaga całkowitej przebudowy.

Podsumowanie #4

Status	❌ nie nadaje się do użycia — aktywnie szkodzi
Taski utworzone	Do 150+/dobę
Znane problemy	1. `status: active` domyślnie. 2. Trigger co 1h bez deduplikacji. 3. Hardcoded `cost > 100`, `globals: []`. 4. Stary format (trigger, filter, create_task, parametry camelCase). 5. Brak `campaign_statuses`. 6. `parent_id: null` na getterze. 7. Zakres dat wczoraj+dziś.
Ocena ogólna	Najgorzej zaimplementowany szablon operacyjnie. Dobra idea, fatalna realizacja.
Rekomendacja	Przebudować od zera wzorując się na `ms_budget_protector`: nowy format, `status: inactive`, konfigurowalny próg, interval 4h, filtr ENABLED.

Test #5 — `budget_deviation` *(Google Ads)*

Data testu: 2026-03-30 | **Plik:** `budget_deviation.json`

Węzeł 1 — Trigger

- ✅ Nowy format (`schedule`), `timezone: Europe/Warsaw`
- ❌ **Krytyczny błąd konfiguracji:** `frequency: daily` + `days_of_month: "first_day"` — workflow odpala się TYLKO 1. dnia miesiąca, nie codziennie
- ⚠️ **Sprzeczne czasy:** `start_time: "01:30"` (outer data) vs `time: "20:00"` (repeat) — niejasne który parametr jest wiążący
- ❌ `status: active` — domyślnie aktywny

Węzeł 2 — Getter: `get_google_ads_customer`

- ❌ `connection_id: 652`, `project_id: 909` — hardcoded dane konkretnego użytkownika
- ❌ **GDPR:** `creator.email: lukas.lochocki@sembot.io` w pliku szablonu — dane osobowe w kodzie

Węzeł 3 — Filter (po getterze)

- ❌ `id = "8725857420"` — hardcoded Google Ads Customer ID konkretnego konta
- ❌ Jako szablon bezużyteczny — każdy inny użytkownik potrzebuje innego ID
- ⚠️ `sourceId` na następnym iteratorze wskazuje na getter (1791), nie na filter (1795) — filtr może być efektywnie omijany

Węzły 4–5 — Iterator + `get_google_ads_metrics`

- ✅ `resource: "customer"` — metryki na poziomie całego konta
- ✅ `start_date: NOW().getFirstDayCurrentMonth()`, `end_date: NOW()` — MTD (month-to-date)
- ⚠️ `limit: 10` — powinno być `limit: 1` (jedno konto, jeden wiersz)
- ✅ `{4.id}` — poprawne odwołanie do ID konta

Węzeł 6 — Router + dwie gałęzie filtrów

```
Gałąź A: cost > monthlyDeviation({globals.budget}, {globals.deviation})   → PRZEKROCZONY
Gałąź B: cost < monthlyDeviation({globals.budget}, -{globals.deviation})  → NIEDOWYDANY
```

- ✅ `monthlyDeviation()` — oblicza oczekiwane wydatki MTD ± tolerancja %
- ✅ Obsługa obu przypadków (over/under) — kompletny monitoring
- ⚠️ `router: data: []` — pusta konfiguracja routera

Węzły 7–8 — `create_task` (obie gałęzie)

✅ Bardzo bogaty opis: rzeczywiste wydatki MTD, oczekiwane, odchylenie %, pozostały budżet, rekomendowany dzienny budżet.

❌ **Dzielenie przez zero na ostatni dzień miesiąca:** `divide(subtract(budget, cost), subtract(getDaysInMonth(), getDayOfMonth()))` → na 31. dnia = 0 w mianowniku.

Dane testowe (budget 55 000 PLN, tolerancja 10%, 15. dzień miesiąca):

Scenariusz	Wydatki MTD	Oczekiwane (50%)	monthlyDev +10%	monthlyDev -10%	Gałąź
Tempo idealne	27 500	27 500	30 250	24 750	brak alertu
Przyspieszony	33 100	27 500	30 250	24 750	✅ A (przekroczony)
Spowolniony	21 000	27 500	30 250	24 750	✅ B (niedowydany)
W tolerancji	29 000	27 500	30 250	24 750	brak alertu

Ocena eksperta SEM

Główna luka — trigger tylko 1. dzień miesiąca: Budget deviation to alert który powinien działać CODZIENNIE. Jeśli konto przepali budżet 15. dnia, workflow nie zareaguje — nie odpal się już do końca miesiąca. Sprzeczne z koncepcją monitoringu budżetu.

Wartościowa logika: `monthlyDeviation()` to poprawna implementacja "budget pacing" — koncepcja używana przez Google Ads Smart Bidding i zewnętrzne narzędzia PPC.

Brak drilldownu: Monitoring na poziomie customer nie wskazuje które kampanie odpowiadają za przekroczenie.

Podsumowanie #5

Status	⚠️ logika poprawna, ale nie działa jako codzienny monitoring
Typ pliku	❌ Realny workflow użytkownika (nie szablon) — GDPR, hardcoded IDs
Znane problemy	1. Trigger tylko 1. dzień miesiąca (fatal). 2. Hardcoded customer ID. 3. Dzielenie przez zero na ostatni dzień miesiąca. 4. GDPR: email autora + IDs. 5. Redundantne iteratory ×2. 6. `limit: 10` zamiast 1. 7. Sprzeczne czasy w triggerze.
Ocena ogólna	Najlepsza logika analityczna w zestawie (MTD pacing z pasmem tolerancji), fatalna implementacja templatingu. To konkretny workflow konkretnego użytkownika, nie szablon.
Rekomendacja	Zmienić trigger na `frequency: daily` bez `days_of_month`. Usunąć GDPR dane. Zastąpić hardcoded IDs przez `null` / `globals`. Dodać guard przed /0 na ostatni dzień miesiąca.

Test #6 — `visibility_drop_alert` *(SEO Monitoring)*

Data testu: 2026-03-30 | **Plik:** `visibility_drop_alert.json`

Węzeł 1 — Trigger

- ❌ Stary format (`trigger_type: cron`, `cron: "0 8 1"`) — poniedziałki 8:00 UTC
- ❌ Brak `timezone` — UTC nie Europe/Warsaw
- ❌ `status: active` domyślnie aktywny

Węzeł 2 — Getter: `get_phrase_monitor_competitors`

- ✅ Nowy typ gettera — dane z monitoringu fraz SEO
- ❌ `project_id: ""` — pusty string
- ❌ `parent_id: null` — węzeł odłączony od triggera (powinno być `parent_id: 1`)

Węzeł 3 — Filtr (stary format)

- ❌ Stary format filtra (`flow_type: "filter"`, stara struktura warunków)
- Warunki: `isMyResult = "true"` [AND?] `rankAbsolute > 5`
- ❌ Brak `logical_operator` między warunkami — AND czy OR? Nieokreślone
- ⚠️ `"true"` jako string — porównanie stringa z booleanem może nie zadziałać

Węzły 4–5 — Iterator + `create_task`

- ❌ Iterator i action w starym formacie
- ⚠️ Minimalny opis taska: tylko keyword, pozycja, domena, URL — brak danych historycznych

Krytyczna luka logiczna

To nie jest "drop alert" — warunek `rankAbsolute > 5` wykrywa aktualne pozycje poniżej 5, nie SPADKI. Fraza permanentnie na pozycji 8 generuje task co tydzień bez końca. Prawdziwy drop alert wymaga porównania current vs previous week.

Dane testowe:

Fraza	isMyResult	rankAbsolute	Przechodzi?
"sembot"	true	2	❌
"narzędzia sem"	true	7	✅ (permanentna pozycja 7 — fałszywy alarm)
"automatyzacja reklam"	false	3	❌
"google ads tool"	true	12	✅ (może być zawsze — fałszywy alarm)

Ocena eksperta SEM

`get_phrase_monitor_competitors` to ciekawy getter (SEMrush/własny monitor SEO), ale użyty błędnie. Brak filtra na wolumen wyszukiwań — frazy z 10 szukań/mies. na poz. 6 dostają taki sam alert jak frazy z 10 000 szukań/mies. SEO monitoring bez wolumenu jest bezużyteczny.

Podsumowanie #6

Status	❌ nie działa jako "drop alert" — logika wykrywa złe pozycje, nie spadki
Znane problemy	1. Stary format (trigger, filter, iterator, action). 2. Logika nie wykrywa spadku (brak dual-getter). 3. `parent_id: null` na getterze. 4. Fałszywe alarmy dla fraz permanentnie poza top 5. 5. `globals: []`. 6. `isMyResult = "true"` string vs boolean. 7. Brak filtra na wolumen. 8. Brak timezone.
Ocena ogólna	Interesująca koncepcja SEO-monitoringu, kompletnie błędna implementacja logiki drop-detection. Całkowita przebudowa wymagana.
Rekomendacja	Przebudować: dual-getter (bieżący + poprzedni tydzień), filtr na percentageChange(rank), dodać wolumen do warunku i opisu taska. Nowy format.

Test #7 — `price_competitiveness_alert` *(Google Merchant Center)*

Data testu: 2026-03-30 | **Plik:** `price_competitiveness_alert.json`

Węzeł 1 — Trigger

- ✅ Nowy format, `timezone: Europe/Warsaw`
- ❌ **Ten sam błąd co #5:** `frequency: daily` + `days_of_month: "first_day"` → odpala się TYLKO 1. dnia miesiąca
- ❌ Sprzeczność z opisem szablonu: "Dzienny alert" vs trigger miesięczny

Węzeł 2 — Getter: `get_google_merchant_center_price_competitiveness_info`

- ✅ Nowy format, GMC Price Competitiveness Report
- ✅ `connection_id: null`, `project_id: null` — poprawne dla szablonu
- ✅ `status: inactive`

Węzeł 3 — Filtr

```
(1) priceCompetitivenessBenchmarkPriceMicros > 0      [AND]   ← guard: benchmark istnieje
(2) productViewPriceMicros > multiply(benchmark, 1.10) [OR]   ← nasza cena > benchmark × 1.10
```

- ✅ Guard `benchmarkPriceMicros > 0` — chroni przed fałszywymi alarmami dla produktów bez benchmarku
- ⚠️ `logical_operator: "OR"` na ostatnim warunku — mylący wzorzec jak w #1, #2

Węzły 4–5 — Iterator + `create_task`

- ✅ Nowy format, `sourceId: 3`
- ✅ Bogaty opis: produkt, marka, kraj, cena, benchmark, próg, 3 sugerowane działania
- ⚠️ Ceny wyświetlane w mikrosach bez konwersji (`÷ 1 000 000`) — user widzi `10000000 PLN` zamiast `10.00 PLN`
- ⚠️ `{subtract({globals.price_premium_threshold},1)}×100%` = `0.1×100%` zamiast czytelnego `10%`

Dane testowe (próg 1.10):

Produkt	Nasza cena (µ)	Benchmark (µ)	Przechodzi?
Produkt A	55 000 000	50 000 000	❌ (55M = 55M, nie >)
Produkt B	66 000 000	50 000 000	✅ (66M > 55M)
Produkt C	45 000 000	50 000 000	❌
Produkt D	30 000 000	0	❌ (guard)

Ocena eksperta SEM

Wartościowy use case: GMC Price Competitiveness Report to realne narzędzie — produkty droższe od benchmarku tracą wyświetlenia w Google Shopping. **Brak filtra priorytetyzacji:** przy dużym katalogu może generować dziesiątki tasków. Brak filtra na minimum impressions, kategoria, lub minimalne odchylenie cenowe.

Podsumowanie #7

Status	⚠️ logika poprawna, trigger miesięczny zamiast dziennego
Znane problemy	1. Trigger tylko 1. dzień miesiąca (sprzeczny z opisem). 2. Ceny w mikrosach bez konwersji. 3. `logical_operator: "OR"` na ostatnim warunku. 4. Formatowanie odchylenia (`0.1×100%`). 5. Brak priorytyzacji (wolumen, kategoria).
Ocena ogólna	Dobry szablon GMC z poprawnym guardem na brak benchmarku. Główna wada: trigger bug (monthly vs daily) i brak konwersji mikros → waluta.
Rekomendacja	Usunąć `days_of_month: "first_day"`. Dodać `divide(..., 1000000)` do wyświetlanych cen. Rozważyć filtr na minimum impressions.

Test #8 — `inventory_aware_alert` *(Google Merchant Center)*

Data testu: 2026-03-30 | **Plik:** `inventory_aware_alert.json`

Węzeł 1 — Trigger

- ❌ Stary format (`trigger_type: cron`, `cron: "0 8 *"`) — brak timezone
- ❌ `status: active` domyślnie aktywny

Węzeł 2 — Getter: `get_google_merchant_center_top_products`

- ❌ `connection_id: ""`, `project_id: ""` — puste stringi
- ❌ `parent_id: null` — węzeł odłączony od triggera

Węzeł 3 — Filtr (stary format)

```
(1) conversions > 5           ← bestseller (hardcoded)
(2) availability contains "out_of_stock"  ← brak w magazynie
```

- ❌ Stary format, brak `logical_operator`
- ⚠️ `conversions > 5` hardcoded — niekonfigurowalne
- ⚠️ **Krytyczny: format availability:** GMC zwraca `"out of stock"` (spacja), nie `"out_of_stock"` (podkreślnik) — filtr prawdopodobnie nie łapie żadnego produktu

Węzły 4–5 — Iterator + `create_task`

- ❌ Stary format
- ⚠️ `{4.price}` w mikrosach bez konwersji
- ✅ Sugeruje akcję: "Pause ads or restock urgently"

Dane testowe:

Produkt	conversions	availability	Przechodzi?
Nike Air Max	42	"out of stock" (spacja)	❌ błąd formatu
Samsung TV	18	"out_of_stock" (podkreślnik)	✅
iPhone 15	28	"in stock"	❌

Ocena eksperta SEM

Świetna koncepcja — reklamy na niedostępne bestsellery to klasyczny problem e-commerce (przepalony budżet, złe UX). **Krytyczny błąd formatu availability** — jeśli GMC zwraca `"out of stock"` ze spacją, cały workflow jest martwy. Brak powiązania z kampaniami (task nie wskazuje które kampanie reklamują produkt).

Podsumowanie #8

Status	❌ potencjalnie martwy — błąd formatu availability
Znane problemy	1. Stary format (trigger, filter, iterator, action). 2. Availability `"out_of_stock"` vs `"out of stock"`. 3. `parent_id: null` na getterze. 4. `conversions > 5` hardcoded. 5. `globals: []`. 6. Ceny w mikrosach. 7. Brak timezone.
Ocena ogólna	Dobra koncepcja, fatalna implementacja. Prawdopodobnie nie wykrywa żadnych produktów.
Rekomendacja	Zweryfikować format `availability` w GMC API. Przebudować na nowy format. Dodać `conversions_threshold` do globals.

Test #9 — `non_converting_keywords` *(Google Ads)*

Data testu: 2026-03-30 | **Plik:** `non_converting_keywords.json`

Węzeł 1 — Trigger

- ❌ Stary format, `cron: "0 8 1"`, brak timezone
- ❌ `status: active` domyślnie aktywny

Węzeł 2 — Getter: `get_google_ads_performance`

- `resource: "keyword_view"` — poprawny resource dla słów kluczowych
- ❌ `startDate` / `endDate` camelCase zamiast `start_date` / `end_date` — może nie działać
- ❌ `connection_id: ""`, `project_id: ""` — puste stringi
- ❌ `parent_id: null` — węzeł odłączony od triggera
- ✅ 60-dniowe okno — statystycznie sensowne

Węzeł 3 — Filtr (stary format)

```
(1) cost > 20           ← hardcoded, niekonfigurowalne
(2) conversions < 1     ← zero konwersji
```

- ❌ Stary format, brak `logical_operator`
- ⚠️ Brak filtra `campaign_status: ENABLED`

Węzły 4–5 — Iterator + `create_task`

- ❌ Stary format
- ⚠️ `{4.title}` — w `keyword_view` pole prawdopodobnie to `keyword_text` nie `title` — task może pokazywać `null`
- Minimalny opis: brak Quality Score, match type, sugerowanej akcji

Dane testowe (cost > 20, conv = 0, 60 dni):

Keyword	cost (60d)	conversions	Przechodzi?
"buty sportowe"	145 PLN	0	✅
"adidas"	67 PLN	3	❌
"kup buty"	12 PLN	0	❌
"sklep"	89 PLN	0	✅

Ocena eksperta SEM

Poprawna koncepcja. Kontekst który brakuje: match type (broad natural ma niższy conv rate), upper-funnel keywords (brandowe nie konwertują bezpośrednio), Quality Score (wysoki QS + 0 conv = problem landing page, nie keyword). Błąd `startDate` camelCase powtarza się (był w `budget_protector`) — systemowy problem spójności API.

Podsumowanie #9

Status	❌ stary format, `startDate` camelCase może uniemożliwiać pobieranie danych
Znane problemy	1. Stary format. 2. `startDate` / `endDate` camelCase. 3. `parent_id: null` na getterze. 4. `cost > 20` hardcoded. 5. `{4.title}` błędna nazwa pola. 6. Brak filtra `campaign_status`. 7. `globals: []`.
Ocena ogólna	Poprawna koncepcja, wiele błędów implementacyjnych. Może generować false positives dla upper-funnel keywords.
Rekomendacja	Nowy format. `start_date` / `end_date`. `cost_threshold` do globals. Dodać QS i match type do opisu. Zweryfikować nazwę pola keyword.

Test #10 — `trending_search_terms` *(Google Ads)*

Data testu: 2026-03-30 | **Plik:** `trending_search_terms.json`

Węzeł 1 — Trigger

- ❌ Stary format, `cron: "0 8 1"`, brak timezone, `status: active`

Węzeł 2 — Getter: `get_google_ads_performance` (`search_term_view`)

- ✅ `resource: "search_term_view"`, `limit: 20`
- ❌ `startDate` / `endDate`, `sortField` / `sortOrder` camelCase — sortowanie i daty prawdopodobnie nie działają
- ❌ `connection_id: ""`, `project_id: ""`, `parent_id: null`

Węzeł 3 — Filtr (stary format)

- `impressions > 100` hardcoded, brak logical_operator

Węzły 4–5 — Iterator + `create_task`

- ❌ Stary format, `{4.title}` błędna nazwa pola (powinno być `search_term`)

Krytyczna luka logiczna

Brak filtra "już jest słowem kluczowym": Search Terms Report zawiera frazy już dodane jako keywords (status: `ADDED`). Bez filtra `status != ADDED` generuje taski dla istniejących keywords — główna wartość szablonu ("keyword expansion") jest nieosiągalna. Brak deduplikacji: te same frazy co tydzień.

Dane testowe:

Fraza	impressions	Status w koncie	Przechodzi?
"buty sportowe"	8400	ADDED (już keyword)	✅ — niepotrzebny
"adidas running"	3200	— (nowa)	✅
"buty"	890	EXCLUDED (negative)	✅ — niepotrzebny

Podsumowanie #10

Status	❌ stary format + luka logiczna (brak filtra na nowe frazy)
Znane problemy	1. Stary format. 2. camelCase params (startDate, sortField, sortOrder). 3. `parent_id: null`. 4. Brak filtra `status != ADDED`. 5. Brak deduplikacji tygodniowej. 6. `{4.title}` błędna nazwa pola. 7. `globals: []`.
Ocena ogólna	Dobra koncepcja keyword expansion, bezużyteczna bez filtra nowych fraz. Sortowanie prawdopodobnie nie działa.
Rekomendacja	Nowy format. Dodać filtr `status` = nowe frazy. Deduplikacja (unikalny tytuł taska). Nowy format params.

Test #11 — `conversion_comparison` *(Google Ads)*

Data testu: 2026-03-30 | **Plik:** `conversion_comparison.json`

Węzeł 1 — Trigger

- ✅ Nowy format
- ❌ `timezone: "Africa/Abidjan"` — błędna strefa (UTC+0 zamiast Europe/Warsaw)
- ❌ `frequency: "hourly"` + `days_of_month: "first_day"` — 24×/dobę ale tylko 1. dnia miesiąca
- ❌ `status: active`, GDPR: email + hardcoded IDs

Węzły 2–5 — dual getter (30d + poprzednie 30d)

- ✅ Dual getter: Node 3 (30d), Node 5 (poprzednie 30d)
- ✅ `start_date` / `end_date` snake_case
- ✅ `campaign_statuses: ["ENABLED"]`

Węzeł 9 — Filtr

```
percentageChange(current_30d_conversions, previous_30d_conversions) < -5
```

- ✅ Nowy format, poprawna logika trendu
- ❌ `conversion_cost: 100` global zdefiniowany ale NIEUŻYWANY w filtrze
- ❌ Brak guardu `previous_conversions > 0` → false alarm lub błąd dla kampanii z 0 konwersji

Węzeł 6 — `create_task`

- ✅ Bogaty opis: oba okresy, konwersje, koszt, CTR
- ❌ Opis zawiera fałszywą informację: "koszt <100 PLN" jako warunek — filtr tego nie sprawdza

Dane testowe:

Kampania	conv (30d)	conv (prev 30d)	percentageChange	Przechodzi?
Brand Exact	45	48	-6.25%	✅
Non-Brand	29	25	+16%	❌
Display	0	3	-100%	✅ (fałszywy alarm)

Podsumowanie #11

Status	⚠️ działa, ale trigger godzinny tylko 1. dnia miesiąca + nieużywany global
Typ pliku	❌ Realny workflow użytkownika — GDPR, hardcoded IDs
Znane problemy	1. Zła timezone (Africa/Abidjan). 2. Hourly + first_day. 3. `conversion_cost` global nieużywany. 4. Brak guardu previous_conversions > 0. 5. Fałszywy opis warunków. 6. Redundantny iterator. 7. GDPR.
Ocena ogólna	Najlepiej skonstruowana logika dual-30d porównawcza. Wiele błędów konfiguracyjnych — realny workflow, nie szablon.
Rekomendacja	Zmienić timezone. Trigger tygodniowy. Dodać cost do filtra. Guard previous_conversions > 0. Poprawić opis warunków.

Test #12 — `ai_weekly_report` *(Google Ads + AI Agent)*

Data testu: 2026-03-30 | **Plik:** `ai_weekly_report.json`

Węzeł 1 — Trigger

- ❌ Stary format, `cron: "0 8 1"`, brak timezone, `status: active`, `globals: []`

Węzeł 2 — Getter: `get_google_ads_performance` (`resource: campaign`, 7 dni)

- ❌ `startDate` / `endDate` camelCase
- ❌ `connection_id: ""`, `project_id: ""`, `parent_id: null`

Węzeł 3 — `ai_agent` (NOWY TYP WĘZŁA)

- `agent_id: null` — brak skonfigurowanego agenta
- ❌ **Brak iteratora:** `{2.campaignName}` w AI promptcie przy array — AI dostanie pierwszą kampanię lub zdeformowane dane
- ❌ `CPA: {2.cost}/{2.conversions}` — literalny slash, nie obliczenie (AI widzi `"CPA: 1234.56/15"`)
- ⚠️ Prompt bazowy: brak impressions, ROAS, budget utilization

Węzeł 4 — `create_task`

- ❌ Stary format
- `description: "{3.output}"` — wynik AI jako opis taska
- ⚠️ Format `{3.output}`: Markdown może nie renderować się w tasku

Ocena eksperta SEM

Ciekawa architektura Getter → AI → Task: Jedna syntetyczna analiza to lepsze UX niż 20 osobnych tasków. Koncepcja właściwa. **Implementacja broken:** brak iteratora + `{2.campaignName}` bez serializacji = AI dostaje dane pierwszej kampanii. Żeby AI przeanalizował wszystkie kampanie, należy użyć customer-level gettera lub właściwej serializacji tablicy.

Podsumowanie #12

Status	❌ stary format + brak iteratora = niekompletne dane dla AI
Znane problemy	1. Stary format. 2. `startDate` / `endDate` camelCase. 3. Brak iteratora przy `{2.campaignName}`. 4. `CPA: cost/conv` text zamiast kalkulacji. 5. `agent_id: null`. 6. `globals: []`. 7. `parent_id: null`.
Ocena ogólna	Innowacyjna koncepcja AI synthesis, broken implementacja.
Rekomendacja	Użyć customer-level gettera lub dodać iterator+agregację. Poprawić CPA. `agent_id`. Nowy format.

Test #13 — `campaign_intelligence_scoring` *(Google Ads + AI Agent)*

Data testu: 2026-03-30 | **Plik:** `campaign_intelligence_scoring.json`

Struktura identyczna z #12 — Getter → AI Agent → create_task.

Różnice względem #12

- ✅ Okno 30 dni (vs 7 dni) — bardziej istotne statystycznie
- ✅ Lepszy prompt: AI ocenia każdą kampanię 1–10 z Strengths/Weaknesses/Priority Action
- ❌ `ROAS: {2.conversions_value}/{2.cost}` — slash zamiast kalkulacji (identyczny błąd jak CPA w #12)

Znane problemy (identyczne z #12)

❌ Stary format, `startDate` / `endDate` camelCase, `parent_id: null`, brak iteratora (AI dostaje dane jednej kampanii), ROAS/CPA jako text/slash, `agent_id: null`, `globals: []`, `status: active`.

Podsumowanie #13

Status	❌ stary format + brak iteratora = niekompletne dane dla AI
Znane problemy	Identyczne z #12 + `ROAS: conv_value/cost` slash
Ocena ogólna	Lepsza koncepcja promptu (scoring z rubryką 1-10), identyczne błędy. Duplikat architektury #12.
Rekomendacja	Połączyć #12 i #13 w jeden szablon z konfigurowalnym trybem. Poprawić serializację danych do AI.

Test #14 — `pmax_analysis` *(Google Merchant Center + AI Agent)*

Data testu: 2026-03-30 | **Plik:** `pmax_analysis.json`

Węzeł 1 — Trigger

- ❌ Stary format, `cron: "0 8 1"`, brak timezone, `status: active`, `globals: []`

Krytyczny błąd struktury

```
Node 1 (trigger)   parent_id: null  ← poprawne
Node 2 (getter 1)  parent_id: null  ← ❌ odłączony
Node 3 (getter 2)  parent_id: null  ← ❌ odłączony
Node 4 (ai_agent)  parent_id: null  ← ❌ odłączony od obu getterów
Node 5 (create)    parent_id: 4     ← ✅ połączony z AI
```

❌ **4 węzły z parent_id: null — cały workflow rozłączony. Nie wykona się.**

Węzły 2–3 — dwa gettery GMC

- Node 2: `get_google_merchant_center_performance_info` (30 dni), ❌ camelCase params
- Node 3: `get_google_merchant_center_top_products`, ❌ puste IDs
- ⚠️ `{2.title}` — w GMC performance brak pola "title"
- ⚠️ `{3.price}` — ceny w mikrosach

Węzeł 4 — `ai_agent` (parent_id: null)

- Prompt próbuje połączyć `{2.xxx}` + `{3.xxx}` — ale AI nie jest połączony z getterami

Ocena eksperta SEM

Interesująca koncepcja cross-source (GMC performance + top products). AI mogłoby korelować produkty z wysokim CTR i niską konwersją z dostępnością. **Workflow niekompletny** — parent_id nie zostały ustawione. Brak mechanizmu merge/join dla dwóch równoległych getterów.

Podsumowanie #14

Status	❌ workflow rozłączony — nie wykona się
Znane problemy	1. 4 węzły z `parent_id: null`. 2. Stary format. 3. camelCase params. 4. `agent_id: null`. 5. `{2.title}` błędne pole. 6. Brak merge node dla dwóch getterów. 7. `globals: []`.
Ocena ogólna	Ambitna koncepcja cross-source, workflow niekompletny i rozłączony.
Rekomendacja	Połączyć węzły (parent_id). Trigger → getter1 → getter2 → AI (sekwencja). Nowy format.

Test #15 — `seasonal_ad_copy_check` *(Google Ads + AI Agent)*

Data testu: 2026-03-30 | **Plik:** `seasonal_ad_copy_check.json`

Węzeł 1 — Trigger

- ❌ Stary format, brak timezone, `status: active`, `globals: []`
- ✅ `cron: "0 8 1 "` — poprawny miesięczny cron (1. dzień miesiąca)

Węzeł 2 — Getter: `get_google_ads_ad_copy`

- ❌ `parent_id: null` — odłączony od triggera
- ❌ `connection_id: ""`, `project_id: ""`

Węzły 3–4 — Filtr (adStatus=ENABLED) + Iterator

- ✅ `adStatus = "ENABLED"` — tylko aktywne reklamy
- ✅ Iterator przed AI agentem — każda reklama osobna analiza (poprawny wzorzec!)

Węzeł 5 — `ai_agent` (per reklama)

- ✅ Dostaje: campaignName, headline1/2/3, description1/2, finalUrl
- ⚠️ RSA headlines w Google Ads API są tablicą, nie `headline1` — pola mogą być null
- ⚠️ Skalowalność: 500 aktywnych reklam = 500 AI wywołań + 500 tasków/miesiąc

Podsumowanie #15

Status	⚠️ koncepcja poprawna, stary format, problem skalowalności
Znane problemy	1. Stary format. 2. `parent_id: null` na getterze. 3. `headline1/2/3` może nie istnieć w DTO. 4. 500 reklam = 500 AI tasków/miesiąc. 5. Brak filtra impressions > 0. 6. `globals: []`. 7. `agent_id: null`.
Ocena ogólna	Pierwszy szablon z poprawnym Iterator → AI agentem. Dobra koncepcja monthly ad copy review, problem skalowalności i nazwy pól RSA.
Rekomendacja	Dodać filtr `impressions > min`. Zweryfikować nazwy pól RSA. Nowy format. Ograniczyć do N reklam per kampania.

Test #16 — `daily_ppc_brief` *(Google Ads + AI Agent)*

Data testu: 2026-03-30 | **Plik:** `daily_ppc_brief.json`

Węzeł 1 — Trigger

- ✅ Nowy format, `timezone: Europe/Warsaw`, `status: inactive`, `globals: {}`
- ❌ **Ten sam bug co #5, #7:** `frequency: daily` + `days_of_month: "first_day"` → odpala się TYLKO 1. dnia miesiąca, nie codziennie

Węzeł 2 — Getter + Węzeł 3 — Filtr

- ✅ `parent_id: 1` — poprawnie połączony z triggerem
- ✅ Nowy format filtra: `cost > 0`
- ❌ `startDate` / `endDate` camelCase

Węzeł 4 — `ai_agent`

- ❌ Brak iteratora: `{2.campaignName}` przy tablicy
- ✅ **Najlepszy prompt w zestawie:** konkretny, 150 słów max, prosi o 1 konkretną akcję na dziś
- ⚠️ `{2.roas}` może być null w DTO
- ❌ `agent_id: null`

Węzeł 5 — `create_task`

- ✅ Nowy format (`title`, `project_id: null`)

Podsumowanie #16

Status	⚠️ działa 1 raz/miesiąc zamiast codziennie (trigger bug)
Znane problemy	1. `days_of_month: "first_day"`. 2. Brak iteratora przed AI. 3. `startDate` / `endDate` camelCase. 4. `{2.roas}` może być null. 5. `agent_id: null`.
Ocena ogólna	Najlepszy prompt AI w zestawie, nowy format, inactive. Fatalne przez trigger bug.
Rekomendacja	Usunąć `days_of_month: "first_day"`. Dodać iterator. `start_date` / `end_date`. `agent_id`.

Test #17 — `weekly_opportunity_scout` *(Google Ads + AI Agent)*

Data testu: 2026-03-30 | **Plik:** `weekly_opportunity_scout.json`

Węzeł 1 — Trigger

- ✅ Nowy format, `timezone: Europe/Warsaw`, `status: inactive`
- ✅ `frequency: weekly`, `days_of_week: ["monday"]` — poprawny tygodniowy trigger (brak first_day buga!)
- ✅ `globals: { min_impressions: 50 }` — konfigurowalny

Węzły 2–3 — Getter + Filtr

- ✅ `parent_id: 1`, `resource: "search_term_view"`, `limit: 100`
- ❌ `startDate` / `endDate`, `sortField` / `sortOrder` camelCase
- ✅ Nowy format filtra: `impressions > {globals.min_impressions}`

Węzeł 4 — `ai_agent`

- ❌ `{2.title}` — powinno być `search_term` (błąd nazwy pola)
- ✅ Doskonały prompt: grupowanie fraz, rekomendacja match type, competitors, structured output
- ❌ `agent_id: null`

Węzeł 5 — `create_task`

- ✅ Nowy format, `project_id: null`

Podsumowanie #17

Status	⚠️ prawie dobry — camelCase params i błędna nazwa pola
Znane problemy	1. camelCase params (startDate, sortField, sortOrder). 2. `{2.title}` → powinno być `search_term`. 3. `agent_id: null`. 4. `logical_operator: "OR"`.
Ocena ogólna	Technicznie najlepszy AI szablon: inactive, globals, nowy format, poprawny trigger, doskonały prompt. Drobne błędy techniczne.
Rekomendacja	`start_date` / `end_date`, `sort_field` / `sort_order`. Zmienić `{2.title}` na `{2.search_term}`. `agent_id`.

Test #18 — `search_terms_clustering` *(Google Ads + AI Agent)*

Data testu: 2026-03-30 | **Plik:** `search_terms_clustering.json`

Węzeł 1 — Trigger

- ✅ Nowy format, `status: inactive`, `timezone: Europe/Warsaw`
- ✅ `frequency: "monthly"`, `days_of_month: "first_day"` — poprawne dla miesięcznego triggera
- ✅ `globals: { min_impressions: 30 }`

Węzły 2–3 — Getter + Filtr

- ✅ `parent_id: 1`, `resource: "search_term_view"`, `limit: 200`
- ❌ camelCase: `startDate`, `endDate`, `sortField`, `sortOrder`
- ✅ Nowy format filtra: `impressions > {globals.min_impressions}`

Węzeł 4 — `ai_agent`

- ⚠️ `{2.title}` → powinno być `search_term`
- ✅ Doskonały prompt: klastry semantyczne, nazwy, total impressions, próg "własna ad group"
- ❌ `agent_id: null`

Podsumowanie #18

Status	⚠️ prawie dobry — camelCase params i błąd nazwy pola
Znane problemy	1. camelCase (startDate, sortField, sortOrder). 2. `{2.title}` → `search_term`. 3. `agent_id: null`.
Ocena ogólna	Bardzo dobry: inactive, globals, nowy format, poprawny trigger, doskonały prompt do restrukturyzacji ad groups.
Rekomendacja	`start_date` / `end_date`, `sort_field` / `sort_order`. `{2.search_term}`. `agent_id`.

Test #19 — `monthly_cross_platform_summary` *(Google Ads + Microsoft Ads + AI)*

Data testu: 2026-03-30 | **Plik:** `monthly_cross_platform_summary.json`

Węzeł 1 — Trigger

- ✅ Nowy format, `status: inactive`, `timezone: Europe/Warsaw`, `frequency: monthly`

Przepływ

`Trigger → Getter Google → Getter Microsoft → AI Agent → create_task`

- ✅ Poprawny sekwencyjny chain: oba gettery dostępne dla AI
- ⚠️ Google getter: camelCase (`startDate` / `endDate`) | MS getter: snake_case (`start_date` / `end_date`) — niespójność

Węzeł 4 — `ai_agent`

- ✅ Doskonały prompt: 5-sekcyjne podsumowanie cross-platform, liczby, 3 akcje
- ⚠️ `{2.roas}` / `{3.roas}` — ROAS może być null w DTOs
- ⚠️ `{3.campaignName}` — MS Ads może używać innej nazwy pola
- ❌ `agent_id: null`

Podsumowanie #19

Status	✅ najlepiej zarchitekturowany cross-platform szablon
Znane problemy	1. Google getter camelCase. 2. `roas` niepewne. 3. MS field names. 4. `agent_id: null`.
Ocena ogólna	Wzorowy szablon: inactive, nowy format, sekwencyjny dual-getter, doskonały prompt.
Rekomendacja	`start_date` / `end_date` Google getter. Zweryfikować MS field names. `agent_id`.

Test #20 — `keyword_quality_audit` *(Google Ads + AI Agent)*

Data testu: 2026-03-30 | **Plik:** `keyword_quality_audit.json`

- ✅ Nowy format, `status: inactive`, tygodniowy trigger (monday), `globals: { min_cost: 10 }`
- ✅ `parent_id: 1`, nowy format filtra: `cost > {globals.min_cost}`
- ❌ `startDate` / `endDate` camelCase, `{2.title}` → `keyword_text`, `agent_id: null`
- ✅ Dobry prompt: PAUSE CANDIDATES + WATCH LIST, kryteria CPA > 3x, CTR < 0.5%

Status	⚠️ camelCase + błąd nazwy pola
Rekomendacja	`start_date` / `end_date`, `{2.keyword_text}`, `agent_id`

Test #21 — `master_account_audit` *(Google Ads + AI Agent)*

Data testu: 2026-03-30 | **Plik:** `master_account_audit.json`

- ✅ Nowy format, `status: inactive`, tygodniowy trigger
- ✅ Dual getter sekwencyjny: campaigns → search_terms → AI → task
- ❌ Oba gettery camelCase, `{3.title}` → `search_term`, `agent_id: null`
- ✅ Najkonkretniejszy prompt: dokładnie 3 ustalenia, urgency ranking, jedno zdanie + liczby + akcja

Status	⚠️ camelCase + błąd nazwy pola
Rekomendacja	camelCase params, `{3.search_term}`, `agent_id`

Test #22 — `competitor_gap_analysis` *(Phrase Monitor + Google Ads + AI)*

Data testu: 2026-03-30 | **Plik:** `competitor_gap_analysis.json`

- ✅ Nowy format, `status: inactive`, tygodniowy trigger, dual getter (Phrase Monitor + keywords)
- ❌ `startDate` / `endDate` camelCase (Google getter), `{3.title}` → `keyword_text`, `agent_id: null`
- ⚠️ `{2.is_my_result}` vs #6's `isMyResult` — niespójność nazwy pola dla tego samego gettera
- ✅ Doskonały prompt: cross-reference gaps, grupowanie tematyczne, priorytetyzacja komercyjna

Status	⚠️ camelCase + field name issues
Rekomendacja	`start_date` / `end_date`, zweryfikować `is_my_result` vs `isMyResult`, `agent_id`

Test #23 — `low_ctr_detective` *(Google Ads + AI Agent)*

Data testu: 2026-03-30 | **Plik:** `low_ctr_detective.json`

- ✅ Nowy format, `status: inactive`, tygodniowy trigger
- ✅ `globals: { ctr_threshold: 2, min_impressions: 500 }` — dwa parametry
- ✅ Filtr: `impressions > min` **[AND]** `ctr < threshold` — `logical_operator: "AND"` poprawnie
- ❌ `startDate` / `endDate` camelCase, `agent_id: null`
- ⚠️ `ctr_threshold: 2` — jeśli CTR w DTO jest dziesiętne (0.032), filtr zawsze true (wszystkie kampanie mają ctr < 2)
- ✅ Dobry prompt: benchmarki per typ (search 3-5%, display 0.1-0.3%), diagnoza, ranking urgency

Status	⚠️ **camelCase + potencjalny format CTR**
Rekomendacja	Zweryfikować format `ctr` (decimal vs %). `start_date` / `end_date`. `agent_id`.

Test #24 — `ad_copy_quality_scorer` *(Google Ads + AI Agent)*

Data testu: 2026-03-30 | **Plik:** `ad_copy_quality_scorer.json`

- ✅ Nowy format, `status: inactive`, miesięczny trigger, `limit: 500`, `adStatus = ENABLED`
- ❌ `{2.headline1/2/3}`, `{2.description1/2}` — RSA field names mogą nie istnieć w DTO
- ⚠️ 500 reklam w jednym AI call — może przekroczyć context window modelu
- ✅ Doskonały prompt: skala 1-5, 5 kryteriów, 3-poziomowy output (NEEDS REWRITE / WATCH LIST / GOOD)
- ❌ `agent_id: null`

Status	⚠️ RSA field names + `agent_id: null`
Rekomendacja	Zweryfikować RSA field names w DTO. `limit` do globals. `agent_id`.

Test #25 — `ad_copy_message_audit` *(Google Ads + AI Agent)*

Data testu: 2026-03-30 | **Plik:** `ad_copy_message_audit.json`

- Identyczna struktura z #24 (monthly, inactive, batch, `limit: 500`, RSA field names, `agent_id: null`)
- ✅ Prompt skupiony na message alignment: USP, generic copy, landing page disconnect
- ⚠️ Duplikat architektury #24 — różny kąt analizy, ale te same problemy

Status	⚠️ identyczne problemy jak #24
Ocena ogólna	Komplementarny do #24, warto zachować oba — różne aspekty copy review.
Rekomendacja	To samo co #24. Rozważyć scalenie #24 + #25 w jeden szablon z konfigurowalnym trybem.

Test #26 — `ms_cpa_campaign_alert` *(Microsoft Ads)*

Data testu: 2026-03-30 | **Plik:** `ms_cpa_campaign_alert.json`

- ✅ Nowy format, `status: inactive`, `timezone: Europe/Warsaw`, tygodniowy trigger
- ✅ `globals: { cpa_target: 50, min_spend: 100 }`, `start_date`/`end_date` snake_case
- ✅ Filtr 3 warunki: `spend > min_spend` [AND] `conversions > 0` [AND] `costPerConversion > cpa_target` — guard!
- ⚠️ Iterator `sourceId: 2` (getter zamiast filter)
- ⚠️ CPA bez zaokrąglenia w tytule

Status	✅ działa end-to-end
Ocena ogólna	Solidny odpowiednik Google #1. Nowy format, inactive, snake_case, guard konwersje > 0.
Rekomendacja	Zmienić `sourceId` iteratora na filter. `round()` CPA w tytule.

Test #27 — `ms_roas_drop_alert` *(Microsoft Ads)*

Data testu: 2026-03-30 | **Plik:** `ms_roas_drop_alert.json`

- ✅ Nowy format, `status: inactive`, dual getter (7d + poprzednie 7d), `globals: { roas_target: 3.0, min_spend: 100 }`
- ✅ `start_date` / `end_date` snake_case na obu getterach
- ⚠️ Iterator `sourceId: 2` (getter zamiast filter)
- ❌ Brak guardu `revenue > 0` — fałszywe alarmy dla kampanii bez revenue
- ⚠️ Nazwa "Drop Alert" myląca — wykrywa ROAS poniżej celu, nie spadek
- ✅ Opis: oba okresy, `percentageChange(roas_current, roas_prev)`

Status	✅ działa, false alarms dla kampanii bez revenue
Ocena ogólna	Dobry odpowiednik Google #2. Dual getter + percentageChange.
Rekomendacja	`revenue > 0` guard. `sourceId` iteratora. Zmienić nazwę na "ROAS Below Target".

Test #28 — `ms_zero_impression_campaigns` *(Microsoft Ads)*

Data testu: 2026-03-30 | **Plik:** `ms_zero_impression_campaigns.json`

- ✅ Nowy format, `status: inactive`, tygodniowy trigger, `start_date` / `end_date` snake_case
- ✅ `impressions < 1` — prosta logika
- ⚠️ Iterator `sourceId: 2` (getter zamiast filter)
- ✅ Bogaty opis: 5-punktowa lista przyczyn + link do diagnostyki

Status	✅ działa end-to-end
Ocena ogólna	Solidny odpowiednik Google #3. Nowy format, inactive, bogaty opis diagnostyczny.
Rekomendacja	Zmienić `sourceId` iteratora. Dodać wzmiankę o nowych kampaniach.

Test #29 — `ms_budget_protector` *(Microsoft Ads)*

Data testu: 2026-03-30 | **Plik:** `ms_budget_protector.json`

- ✅ Nowy format, `status: inactive`, `globals: { daily_budget_threshold: 500 }`
- ✅ `frequency: hourly`, `interval: 4` — co 4h (znaczna poprawa vs Google #4: co 1h + active)
- ❌ `days_of_month: "first_day"` — hourly + first_day = prawdopodobnie odpala się co 4h TYLKO 1. dnia miesiąca
- ⚠️ Iterator `sourceId: 2` (getter zamiast filter), brak deduplikacji tasków

Status	⚠️ **`days_of_month: "first_day"` — prawdopodobnie martwy 29/30 dni w miesiącu**
Ocena ogólna	Najlepszy budget protector (inactive, globals, 4h). Trigger bug może go dezaktywować.
Rekomendacja	Usunąć `days_of_month: "first_day"`. Rozważyć deduplikację.

Test #30 — `ms_budget_deviation` *(Microsoft Ads)*

Data testu: 2026-03-30 | **Plik:** `ms_budget_deviation.json`

- ✅ Nowy format, `status: inactive`, `globals: { budget: 5000, deviation: 10 }`, snake_case
- ❌ `days_of_month: "first_day"` — trigger bug (monthly zamiast daily)
- ✅ Iterator `sourceId: 3` — wskazuje na FILTR (poprawnie! lepsza implementacja niż #26-#29)
- ❌ Dzielenie przez zero na ostatni dzień miesiąca (identyczny bug co Google #5)
- ⚠️ Per-campaign zamiast account-level (niezgodność z opisem "wydatki konta")
- ⚠️ Tylko gałąź "przekroczony" (brak "poniżej")

Status	⚠️ trigger bug + /0 na ostatni dzień
Ocena ogólna	Lepsza implementacja niż Google #5 (inactive, brak GDPR, poprawny sourceId). Trigger bug i per-campaign mismatch.
Rekomendacja	Usunąć `days_of_month`. Guard przed /0. Dodać gałąź "poniżej".

Test #31 — `ms_non_converting_keywords` *(Microsoft Ads)*

Data testu: 2026-03-30 | **Plik:** `ms_non_converting_keywords.json`

- ✅ Nowy format, `status: inactive`, tygodniowy trigger, `globals: { min_spend: 50 }`
- ✅ `get_microsoft_advertising_keywords` — dedykowany getter keywords
- ✅ snake_case, filter: `spend > min_spend` [AND] `conversions < 1`
- ✅ Iterator `sourceId: 3` — wskazuje na filtr (poprawnie)
- ✅ `{2.keyword}`, `{2.keywordId}`, `{2.campaignName}` — poprawne MS field names
- ✅ Bogaty opis z 4 sugerowanymi akcjami

Status	✅ działa end-to-end
Ocena ogólna	Solidny, znacznie lepszy od Google #9. Inactive, globals, snake_case, poprawny sourceId.

Test #32 — `ms_trending_search_terms` *(Microsoft Ads)*

Data testu: 2026-03-30 | **Plik:** `ms_trending_search_terms.json`

- ✅ Nowy format, `status: inactive`, tygodniowy trigger, `globals: { min_impressions: 100 }`
- ✅ `get_microsoft_advertising_search_terms`, snake_case, iterator `sourceId: 3` (filter)
- ✅ `{2.searchQuery}` — poprawna nazwa pola (vs Google #10: błędnie `{4.title}`)
- ⚠️ Ta sama luka co Google #10: brak filtra na nowe frazy (nie w keyword liście)
- ⚠️ Brak deduplikacji tygodniowej

Status	✅ działa end-to-end
Ocena ogólna	Lepszy od Google #10: poprawna nazwa pola, snake_case, poprawny sourceId. Shared luka logiczna.

Test #33 — `ms_ai_weekly_report` *(Microsoft Ads + AI Agent)*

Data testu: 2026-03-30 | **Plik:** `ms_ai_weekly_report.json`

- ✅ Nowy format, `status: inactive`, tygodniowy trigger, snake_case
- ✅ MS field names bez slash: `{2.spend}`, `{2.roas}`, `{2.costPerConversion}`
- ❌ Brak iteratora przed AI agent, `agent_id: null`

Status	⚠️ brak iteratora — AI dostaje niekompletne dane
Ocena ogólna	Lepszy od Google #12 (MS field names poprawne). Ta sama architektoniczna luka.

Test #34 — `ms_conversion_comparison` *(Microsoft Ads)*

Data testu: 2026-03-30 | **Plik:** `ms_conversion_comparison.json`

- ✅ Nowy format, `status: inactive`, tygodniowy trigger
- ✅ `globals: { conversion_decrease: -5, min_spend: 100 }` — oba używane w filtrze
- ✅ Dual getter 30d + poprzednie 30d, snake_case
- ✅ Filtr: `spend > min_spend` [AND] `percentageChange(conv, prev_conv) < conversion_decrease`
- ⚠️ Iterator `sourceId: 2` (getter zamiast filter)
- ⚠️ Brak guardu `previous_conversions > 0`

Status	✅ działa end-to-end
Ocena ogólna	Wzorowy szablon. Znacznie lepszy od Google #11 (inactive, brak GDPR, weekly, oba globals).
Rekomendacja	`sourceId` iteratora. Guard `{3.conversions} > 0`.

Test #35 — `ms_campaign_intelligence_scoring` *(Microsoft Ads + AI Agent)*

Data testu: 2026-03-30 | **Plik:** `ms_campaign_intelligence_scoring.json`

- ✅ Nowy format, `status: inactive`, snake_case
- ✅ MS field names bez slash: `{2.roas}`, `{2.costPerConversion}`, `{2.revenue}`
- ❌ Brak iteratora przed AI agent, `agent_id: null`

Status	⚠️ brak iteratora
Ocena ogólna	MS port #13. Lepszy od Google (MS field names poprawne). Identyczne architektoniczne błędy.

Test #36 — `ms_seasonal_ad_copy_check` *(Microsoft Ads + AI Agent)*

Data testu: 2026-03-30 | **Plik:** `ms_seasonal_ad_copy_check.json`

- ✅ Nowy format, `status: inactive`, miesięczny trigger (monthly + first_day = poprawne)
- ✅ Iterator → AI agent (per kampania) — poprawna architektura Iterator → AI!
- ✅ Iterator `sourceId: 3` — wskazuje na filtr (poprawnie)
- ❌ `agent_id: null`
- ❌ Brak gettera ad copy — AI ma tylko nazwę kampanii + spend + CTR (bez rzeczywistych tekstów reklam). Analiza sezonowości copy jest niemożliwa bez treści reklam.

Status	⚠️ brak treści reklam — AI analizuje tylko nazwę kampanii
Ocena ogólna	Poprawna architektura (Iterator → AI). Kluczowy problem: AI nie ma dostępu do tekstów reklam.
Rekomendacja	Zastąpić campaigns getter dedykowanym ad copy getterem. `agent_id`.

Test #37 — `microsoft_ads_waste_finder` *(Microsoft Ads)*

Data testu: 2026-03-30 | **Plik:** `microsoft_ads_waste_finder.json`

- ✅ Nowy format, `status: inactive`, tygodniowy trigger, `globals: { min_spend: 50 }`
- ✅ `get_microsoft_advertising_search_terms`, snake_case
- ⚠️ `ms_ads_raw_xml_request: null` — niestandardowy param
- ✅ Filtr: `spend > min_spend` [AND] `conversions < 1` — `logical_operator: "AND"` poprawnie
- ✅ Iterator `sourceId: 3` — wskazuje na filtr (poprawnie)
- ✅ `{4.searchQuery}` — poprawna nazwa pola
- ✅ **Najlepszy opis taska w całym zestawie:** 5 akcji z rozróżnieniem CTR wysoki vs niski → różna diagnoza

Status	✅ działa end-to-end — wzorowy szablon
Ocena ogólna	Najlepszy szablon search terms w zestawie. Inactive, globals, snake_case, poprawny sourceId, bogaty opis.

Test #38 — `ms_ads_winners` *(Microsoft Ads)*

Data testu: 2026-03-30 | **Plik:** `ms_ads_winners.json`

- ❌ **Stary format** (`trigger_type: cron`, `flow_type`, `action_type`) — brak timezone
- ❌ `status: active` domyślnie, `globals: []` array
- ❌ `connection_id: ""`, `project_id: ""`, `parent_id: null` na getterze
- ❌ `startDate` / `endDate` camelCase
- Filtr (stary): `conversions > 2` AND `ctr > 0.05`
- ⚠️ `ctr > 0.05` — niejasny format (decimal 5% vs procent 0.05%)
- ⚠️ `{4.searchTerm}` vs #37's `{4.searchQuery}` — niespójna nazwa pola dla tego samego gettera
- Minimalny opis taska

Status	❌ stary format
Ocena ogólna	Dobra koncepcja (winners do ekspansji Google Ads), fatalna implementacja. Bezpośredni kontrast z #37 (ten sam getter, zupełnie inna jakość).
Rekomendacja	Przebudować wzorując się na #37: nowy format, inactive, globals, snake_case. Zweryfikować `searchTerm` vs `searchQuery`.

Podsumowanie zbiorcze — Double Check v2

Klasyfikacja statusów (38 szablonów)

Status	Szablony	Liczba
✅ działa end-to-end	#1, #3, #26, #28, #31, #32, #34, #37	8
⚠️ działa z zastrzeżeniami	#2, #7, #15, #16, #17, #18, #19, #20, #21, #22, #23, #24, #25, #27, #29, #30, #33, #34, #35, #36	20
❌ nie działa / szkodliwy	#4, #6, #8, #9, #10, #12, #13, #14, #38	9
❌ realny workflow (nie szablon)	#5, #11	2

Kluczowe wzorce błędów (ranking częstotliwości)

Błąd	Liczba szablonów
`startDate` / `endDate` camelCase	12
`days_of_month: "first_day"` przy daily/weekly trigger	10
`globals: []` (array zamiast obiektu)	9
`status: active` domyślnie	8
Stary format (trigger/filter/iterator/action)	8
Iterator `sourceId` wskazuje na getter zamiast filtr	8
`agent_id: null`	10
Brak iteratora przed AI agent (batch data)	6
`parent_id: null` na getterze (odłączony)	6
GDPR w pliku szablonu	2

Najlepsze szablony (wzorce do naśladowania)

- **#37 microsoft_ads_waste_finder** — wzorowa implementacja: inactive, globals, snake_case, poprawny sourceId, bogaty diagnostyczny opis
- **#19 monthly_cross_platform_summary** — wzorowa architektura cross-platform: dual getter sekwencyjny, doskonały prompt
- **#17 weekly_opportunity_scout** — najlepszy AI szablon Google: poprawny trigger, globals, nowy format
- **#34 ms_conversion_comparison** — wzorowy szablon alertowy MS: dual getter, oba globals używane
- **#26 ms_cpa_campaign_alert** — solidny alert z guardem na konwersje > 0

Szablony wymagające najpilniejszej przebudowy

- **#4 budget_protector** — natychmiast szkodliwy: status: active + 150+ tasków/dobę
- **#14 pmax_analysis** — kompletnie rozłączony: 4 węzły z parent_id: null
- **#6 visibility_drop_alert** — logika fundamentalnie błędna: wykrywa pozycje, nie spadki
- **#8 inventory_aware_alert** — prawdopodobnie martwy: availability format mismatch
- **#5 budget_deviation** — GDPR + hardcoded IDs + monthly-only trigger